DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DATABASE MANAGEMENT SYSTEMS (DBMS) PROJECT DOCUMENTATION

FULL-STACK MERN ARCHITECTURE

106 / 106 TESTS PASSING

IIT GUWAHATI CAMPUS MODEL

Smart Hostel Management System

An End-to-End Enterprise Solution for Accommodation Management, Real-Time Vacancy Discovery, Maintenance Lifecycle, Analytics, and Resident Services

PRIMARY DEVELOPER
Shiv Prakash

PROJECT REPOSITORY
shiv-pencia/SmartHostel-

DATABASE ENGINE
MongoDB • Mongoose 9

FRONTEND FRAMEWORK
React 19 • Vite 8

BACKEND FRAMEWORK
Node.js • Express 5

SEEDED RESIDENTS
507 Active Users

Complete Technical Reference Manual • Academic Viva Voce Preparation Guide • October 2026

DOCUMENT INDEX

Table of Contents

Chapter 1: Project Introduction & Objectives — Page 3

Chapter 2: Technology Stack & Package Architecture — Page 5

Chapter 3: Overall Project Architecture & System Design — Page 8

Chapter 4: Complete Project Folder Structure & File Manifest — Page 11

Chapter 5: Database Design, Schema Constraints & MongoDB Modeling — Page 14

Chapter 6: Backend Implementation, Middleware & API Engine — Page 19

Chapter 7: Authentication, Session Management & Security — Page 24

Chapter 8: Frontend Architecture & Component Engineering — Page 28

Chapter 9: Feature-by-Feature Implementation Guide — Page 32

Chapter 10: Complete REST API Documentation Reference — Page 37

Chapter 11: Critical Code Analysis & Algorithmic Breakdown — Page 42

Chapter 12: End-to-End Execution Flows & Sequence Traces — Page 46

Chapter 13: Installation, Configuration & Deployment Guide — Page 49

Chapter 14: Testing Strategy, Quality Assurance & Debugging — Page 52

Chapter 15: Security Architecture, Edge Cases & Limitations — Page 55

Chapter 16: Viva Voce & Technical Interview Preparation (25+ Questions) — Page 58

Chapter 17: Conclusion & Key Learnings — Page 64

Glossary of Technical Terms & Appendices — Page 65

CHAPTER 1

Project Introduction & Objectives

1.1 Project Overview

The **Smart Hostel Management System** is a full-stack, enterprise-grade web application engineered to modernize and digitize the administration of residential campus hostels. Built using a robust **MERN architecture (MongoDB, Express.js, React 19, Node.js)** and bundled with **Vite 8**, the system replaces error-prone manual paper registers and fragmented spreadsheets with a unified, real-time database management platform.

The project addresses four foundational stakeholders within an academic university environment:

- **Students:** Can track their physical bed allocation details, submit maintenance grievances with category and priority parameters, monitor repair status updates, and read official hostel notices.
- **Hostel Wardens:** Oversee block occupancy, track student allocations, assign maintenance work orders to dedicated technical staff, and manage visitor gate pass check-ins and check-outs.
- **Maintenance Staff:** Access an assigned work-order queue, update real-time progress (`in_progress` → `resolved`), and provide timestamped resolution notes.
- **System Administrators:** Retain global authority over all 14 campus hostels, room capacities, physical bed provisioning, resident status mutations, and real-time database integrity audits.

1.2 Problems Solved by the System

✕ Problems with Traditional Systems

- **Double Bed Allocation:** Manual record keeping frequently leads to two students being assigned to the same physical bed.
- **Grievance Black Hole:** Maintenance complaints written on paper registers lack SLA tracking, status updates, or technician accountability.
- **Inaccurate Occupancy Data:** Administrators lack real-time visibility into vacant bed slots versus broken beds under repair.
- **Security Blind Spots:** Campus visitor logs kept in paper books are difficult to audit, search, or reconcile upon exit.

✓ Solutions Delivered by Smart Hostel

- **Partial Unique Database Indexes:** Mathematically prevents double active allocations directly at the database engine level.
- **Audited Ticket Lifecycle:** Enforces state-machine transitions (`pending` → `assigned` → `resolved`) with technician attribution.
- **Live Aggregation Pipelines:** Real-time calculations of vacancy, usable beds, and occupancy percentages across all 14 hostels.
- **Digital Gate Passes:** Instant check-in recording with ID proof logging and 1-click check-out timestamping.

1.3 Motivation & Scope

The primary motivation behind this project is to implement genuine database management principles—including **referential integrity, compound indexing, transactional consistency, atomicity, and multi-stage aggregation pipelines**—within a practical, modern web platform. Rather than building a generic CRUD mock-up, the system was built and tested against the actual campus topology of **Indian Institute of Technology Guwahati (IIT Guwahati)**, modeling all 14 official residential hostels with realistic floor plans, capacities, and resident populations.

1.4 Key Features at a Glance

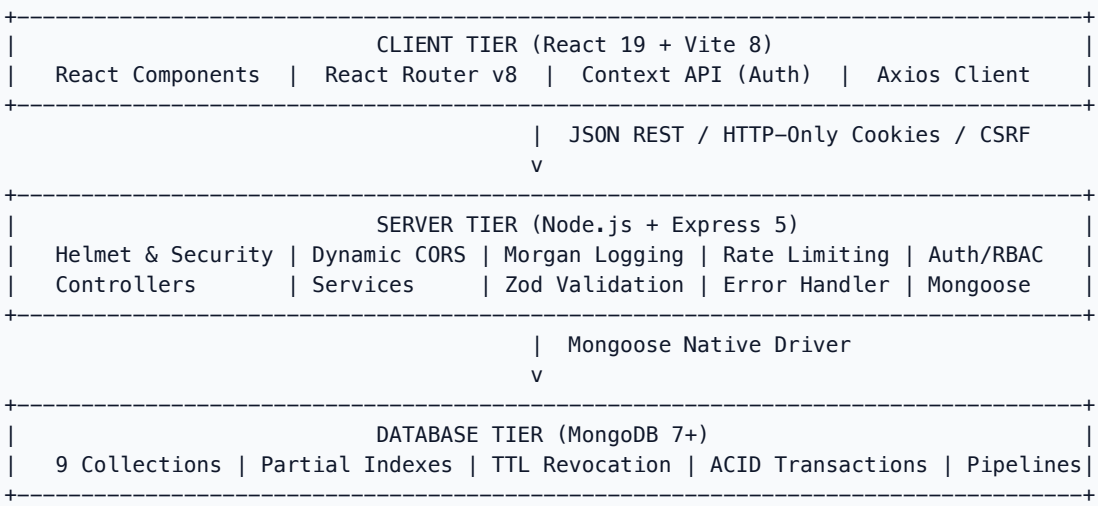
MODULE	PRIMARY CAPABILITIES	DATABASE / TECHNICAL HIGHLIGHTS
Student Allocation	Real-time bed discovery, room selection, student assignment, and 1-click checkout/vacation.	Compound partial unique indexes; MongoDB adaptive transaction sessions.
Grievance Management	Categorized ticket submission, priority tagging, warden technician dispatch, and resolution logging.	State machine constraints, user attribution references, real-time status filtering.
Reports & Analytics	Live occupancy rates %, breakdown by room type, complaint category trends, and database integrity audits.	MongoDB aggregation pipelines (`$lookup` , `$group` , `$map` , `$cond`).
Hostel & Room Inventory	Block management, gender segregation rules, floor definitions, and physical bed provisioning.	Compound uniqueness constraints (`hostel + roomNumber` , `room + bedNumber`).
Notice Board & Visitors	Targeted hostel circulars, event announcements, visitor gate passes, and check-out auditing.	Targeted foreign keys, date-indexed sorting, status enumeration.

CHAPTER 2

Technology Stack & Package Architecture

2.1 Full-Stack Architectural Layers

The system utilizes a decoupled client-server architecture. The frontend is built as a responsive Single Page Application (SPA) communicating over HTTPS/JSON REST APIs with the Express backend, which interfaces directly with MongoDB via the Mongoose Object Data Modeling (ODM) library.



2.2 Backend Dependencies (server/package.json)

PACKAGE NAME	VERSION	ROLE AND IMPLEMENTATION JUSTIFICATION IN PROJECT
`express`	`^5.2.1`	Core HTTP application server framework. Version 5 provides native asynchronous error propagation without requiring third-party wrappers for rejected promises.
`mongoose`	`^9.9.4`	Object Data Modeling (ODM) library for MongoDB. Enforces strict document schemas, type casting, validation hooks (pre-save, pre-validate), and index definitions.
`jsonwebtoken`	`^9.0.3`	Generates and verifies cryptographic HMAC SHA-256 (HS256) JSON Web Tokens containing user subject IDs, UUID tokens, and expiration times for stateless authentication.
`bcryptjs`	`^3.0.3`	Adaptive key derivation function using 12 salt rounds to securely hash resident passwords before persisting to database, mitigating rainbow table attacks.
`zod`	`^4.6.5`	TypeScript-first schema declaration and data validation library. Validates all incoming HTTP request bodies and route parameters before controllers are executed.
`cors`	`^2.8.6`	Cross-Origin Resource Sharing middleware configured with dynamic origin validation supporting `localhost` and `127.0.0.1` with credentials.
`helmet`	`^8.3.0`	Applies critical HTTP security response headers (Content-Security-Policy, X-Frame-Options, Strict-Transport-Security) to prevent clickjacking and XSS.
`cookie-parser`	`^1.4.7`	Parses the HTTP `Cookie` request header and populates `req.cookies`, enabling secure extraction of the `accessToken` cookie.
`express-rate-limit`	`^8.7.0`	Prevents Denial-of-Service (DoS) and brute-force credential stuffing by throttling IP requests within configurable time windows.
`morgan`	`^1.12.0`	HTTP request logger middleware producing clean colored terminal output during development for debugging routes, response times, and status codes.
`dotenv`	`^17.4.2`	Loads environment variables from `.env` file into `process.env` during system bootstrapping.

2.3 Frontend Dependencies (client/package.json)

PACKAGE NAME	VERSION	ROLE AND IMPLEMENTATION JUSTIFICATION IN PROJECT
`react` & `react-dom`	`^19.2.8`	React 19 component library utilizing Concurrent Mode, optimized reconciliation algorithms, and hooks (`useState` , `useEffect` , `useCallback` , `useMemo`).
`vite`	`^8.3.0`	Next-generation frontend build tooling and local dev server powered by native ES modules. Yields production bundles in ~220ms.
`react-router`	`^8.4.0`	Client-side declarative routing engine managing public routes, role-guarded layouts, dynamic navigation redirects, and active route state.
`axios`	`^1.20.0`	Promise-based HTTP client featuring centralized interceptors for CSRF header injection (`X-Requested-With`) and normalized API error parsing.
`lucide-react`	`^1.56.0`	Accessible, modern SVG icon library used across dashboards, status pills, tables, and navigation menus.
`vitest`	`^5.0.4`	Native ESM integration test runner executing component, context, and mocked-API integration test suites in headless JSDOM environments.

CHAPTER 3

Project Architecture & System Design

3.1 System Architecture Overview

The Smart Hostel Management System follows the classic **Model-View-Controller-Service (MVCS)** pattern. This design pattern strictly decouples HTTP transport logic from business execution and database queries, guaranteeing testability, maintainability, and clean code boundaries.

```
+-----------------------------------------------------------------------------------------------------+
|                                      MVCS ARCHITECTURAL FLOW                                        |
+-----------------------------------------------------------------------------------------------------+

 [User Browser]
      |
      | 1. HTTP Request (JSON body, Cookies, CSRF Header)
      v
 [Express Router] (/api/v1/...)
      |
      | 2. Passes through Global Middlewares (Helmet, CORS, RateLimit, CSRF)
      v
 [Route Layer] (e.g. allocation.routes.js)
      |
      | 3. Runs Route Middlewares: authenticate() → authorizeRoles("admin", "warden") → validate(schema
      v
 [Controller Layer] (e.g. allocation.controller.js)
      |
      | 4. Unpacks req.body / req.params & passes sanitized inputs to Service
      v
 [Service Layer] (e.g. allocation.service.js)
      |
      | 5. Executes Business Logic, Adaptive Transactions, Pre-conditions
      v
 [Model Layer] (e.g. allocation.model.js)
      |
      | 6. Interacts with MongoDB Engine (Partial Indexes, Validations, Aggregations)
      v
 [MongoDB Database] (hostel_management)
```

3.2 Request-Response Lifecycle

Every incoming HTTP request traverses a strictly controlled multi-stage security and validation pipeline before reaching business logic:

1. **Security Headers (Helmet):** Injects HTTP headers to disable unsafe scripts and iframe embedding.
2. **CORS Verification:** Confirms the client origin matches trusted development and production domains (`localhost:5173`, `127.0.0.1:5173`).
3. **Rate Limiting:** Enforces an allocation of 3,000 requests per 15 minutes during development (100 in production).
4. **Body & Cookie Parsing:** Parses request bodies up to 10KB (mitigating buffer overflow attacks) and extracts signed cookies.
5. **CSRF Mitigation:** For all mutating HTTP verbs (`POST`, `PUT`, `PATCH`, `DELETE`), verifies that `X-Requested-With === "XMLHttpRequest"`.

6. **Authentication Middleware:** Extracts `accessToken`, verifies JWT signature, checks the `RevokedToken` table, and attaches `req.user`.
7. **Role Authorization Middleware:** Verifies that `req.user.role` belongs to permitted roles (e.g., `authorizeRoles("admin", "warden")`).
8. **Zod Schema Validation:** Validates payload formats, emitting standard `400 Bad Request` messages on failure.
9. **Service Execution:** Executes atomic operations within `withOptionalTransaction`.
10. **Standardized Response / Error Middleware:** Emits formatted JSON via `sendSuccess()` or captures exceptions in `error.middleware.js`.

CHAPTER 4

Complete Project Folder Structure

4.1 Complete Directory Tree

```
smart-hostel/
├── PROGRESS_REPORT.md                 # Executive Markdown progress report
├── Smart_Hostel_DBMS_Progress_Report.pdf # 4-page summary report
├── README.md                          # Master project documentation
│
├── server/                            # BACKEND EXPRESS SERVER
│   ├── package.json                   # Server dependencies & test scripts
│   ├── .env.example                   # Backend environment configuration template
│   ├── src/
│   │   ├── server.js                  # HTTP listener & graceful shutdown handlers
│   │   ├── app.js                     # Express app setup, middlewares & route mounting
│   │   ├── config/
│   │   │   ├── db.js                  # MongoDB Mongoose connection manager
│   │   │   └── env.js                 # Environment variable validation & exports
│   │   ├── models/                    # Mongoose database schemas
│   │   │   ├── user.model.js          # Resident, staff, warden, and admin schema
│   │   │   ├── hostel.model.js        # Hostel block schema with gender policies
│   │   │   ├── room.model.js          # Room inventory with capacity bounds
│   │   │   ├── bed.model.js           # Bed units with operational status
│   │   │   ├── allocation.model.js    # Student-bed allocations & partial unique indexes
│   │   │   ├── complaint.model.js     # Maintenance grievances & ticket lifecycle
│   │   │   ├── notice.model.js        # Official campus notices and announcements
│   │   │   ├── visitor.model.js       # Campus entry gate passes & check-out logs
│   │   │   └── revokedToken.model.js  # Blacklisted JWT tokens with TTL auto-purge
│   │   ├── controllers/               # Request handling & HTTP response formatting
│   │   ├── services/                  # Business logic, aggregations & database queries
│   │   ├── middlewares/               # Auth, RBAC, CSRF, rate-limit & error handlers
│   │   ├── validators/                # Zod request validation schemas
│   │   ├── utils/                     # ApiError, token utils, transactions, response helpers
│   │   ├── seed.js                    # Full project seeder (14 hostels, demo users, records)
│   │   ├── seedUsers.js               # High-scale user generator (500 residents)
│   │   └── seedIITGHostels.js         # Dedicated IIT Guwahati 14-hostel population script
│   └── tests/                         # Native integration test suites (85 tests passing)
│
└── client/                            # FRONTEND REACT APPLICATION
    ├── package.json                   # Client dependencies & Vite scripts
    ├── vite.config.js                 # Vite build & plugin configuration
    ├── vitest.config.js               # Vitest integration test configuration
    ├── index.html                     # SPA single-page entry HTML
    └── src/
        ├── main.jsx                   # React 19 root DOM mount
        ├── App.jsx                    # Application root wrapped in AuthProvider
        ├── api/                       # Central Axios client with CSRF interceptors
        ├── config/                    # Role-specific dashboard navigation manifests
        ├── context/                   # Global AuthContext & cookie session provider
        ├── hooks/                     # Custom React hooks (useAuth)
        ├── layouts/                   # DashboardLayout (sidebar/topbar) & PublicLayout
        ├── pages/                     # HomePage, RoleDashboardPage, NotFoundPage
        ├── routes/                    # AppRoutes, ProtectedRoute, RoleRoute, GuestRoute
        ├── styles/                    # global.css design tokens & dashboard.css layout
        ├── features/                  # Modular domain slices (allocations, complaints, etc.)
        └── test/                      # Vitest component & route integration tests (21 tests)
```

CHAPTER 5

Database Design, Schema Constraints & MongoDB Modeling

5.1 Entity Relationship & Collection Model

The MongoDB database (`hostel_management`) incorporates **9 specialized collections**. Referential integrity is strictly maintained through MongoDB `ObjectId` references and verified by application service-layer pre-condition checks.

5.2 Detailed Collection Schemas

5.2.1 Users Collection (users)

Stores identity, encrypted credentials, access roles, and activity status.

FIELD NAME	BSON TYPE	CONSTRAINTS & VALIDATION	DESCRIPTION
`_id`	ObjectId	Primary Key (auto-generated)	Unique user document identifier.
`name`	String	Required, trim, length: 2–100	Full name of resident or staff member.
`email`	String	Required, unique, lowercase, regex match	User's email used as primary login handle.
`password`	String	Required, minlength: 8, `select: false`	Bcrypt-hashed password (salt rounds: 12).
`phone`	String	Optional, trim, default: `null`	Contact phone number.
`role`	String	Enum: `student`, `warden`, `staff`, `admin`	RBAC permission role. Default: `student`.
`isActive`	Boolean	Default: `true`, Index: true	Account status flag allowing instant revocation.
`lastLoginAt`	Date	Default: `null`	Timestamp of most recent authentication.

5.2.2 Hostels Collection (hostels)

Models physical residential blocks across the campus.

FIELD NAME	BSON TYPE	CONSTRAINTS & VALIDATION	DESCRIPTION
`_id`	ObjectId	Primary Key	Unique hostel identifier.
`name`	String	Required, trim, length: 2–120	Full title (e.g. "Brahmaputra Hostel").
`code`	String	Required, unique index, uppercase, regex	Short unique acronym (e.g. `BRH`, `DIS`).
`location`	String	Required, trim, length: 3–200	Campus zone (e.g. "North Campus, IIT Guwahati").
`genderPolicy`	String	Enum: `male`, `female`, `coed`	Gender segregation policy.
`warden`	ObjectId	Ref: `User`, default: `null`	Foreign key to assigned faculty warden.
`isActive`	Boolean	Default: `true`	Operational status of the building.

5.2.3 Rooms Collection (rooms)

Represents physical rooms within a parent hostel block.

FIELD NAME	BSON TYPE	CONSTRAINTS & VALIDATION	DESCRIPTION
`_id`	ObjectId	Primary Key	Unique room identifier.
`hostel`	ObjectId	Ref: `Hostel`, required, immutable	Parent hostel block reference.
`roomNumber`	String	Required, uppercase, regex	Room door identifier (e.g. "A-101", "102").
`floor`	Number	Required, integer, min: 0, max: 100	Floor level (0 = Ground Floor).
`capacity`	Number	Required, integer, min: 1, max: 20	Maximum student beds allowed in the room.
`bedCount`	Number	Default: 0, integer	Denormalized counter of provisioned physical beds.
`status`	String	Enum: `active`, `maintenance`, `inactive`	Operational status of the room.

5.2.4 Allocations Collection (allocations)

Core DBMS entity recording active and historical student bed allocations.

FIELD NAME	BSON TYPE	CONSTRAINTS & VALIDATION	DESCRIPTION
`_id`	ObjectId	Primary Key	Unique allocation document identifier.
`student`	ObjectId	Ref: `User`, required, immutable	Assigned resident student.
`hostel`	ObjectId	Ref: `Hostel`, required, immutable	Assigned hostel block.
`room`	ObjectId	Ref: `Room`, required, immutable	Assigned room reference.
`bed`	ObjectId	Ref: `Bed`, required, immutable	Assigned physical bed document.
`status`	String	Enum: `active`, `vacated`, default: `active`	Allocation state.
`allocatedAt`	Date	Required, default: `Date.now`	Check-in / assignment timestamp.
`vacatedAt`	Date	Default: `null`	Vacation timestamp (required if status=vacated).
`allocatedBy`	ObjectId	Ref: `User`, required	Administrator who authorized the allocation.
`vacatedBy`	ObjectId	Ref: `User`, default: `null`	Administrator who authorized the checkout.

5.3 Database Indexing Strategy

Database Constraints & Indexes Implemented in Production

- `Hostel.index({ code: 1 }, { unique: true })`: Enforces global hostel code uniqueness.
- `Room.index({ hostel: 1, roomNumber: 1 }, { unique: true })`: Prevents duplicate room numbers inside the same hostel block while permitting identical room numbers across different hostels.
- `Bed.index({ room: 1, bedNumber: 1 }, { unique: true })`: Ensures distinct bed labels within each room.
- `Allocation.index({ student: 1 }, { unique: true, partialFilterExpression: { status: "active" } })`: **Zero Double Allocation:** A student can possess at most one active bed allocation at any given time.
- `Allocation.index({ bed: 1 }, { unique: true, partialFilterExpression: { status: "active" } })`: **Zero Double Occupancy:** A physical bed can be allocated to at most one student at any given time.
- `RevokedToken.index({ expiresAt: 1 }, { expireAfterSeconds: 0 })`: **MongoDB TTL Index:** Automatically purges expired JWT tokens from the collection without requiring scheduled cron jobs.

CHAPTER 6

Backend Implementation, Middleware & API Engine

6.1 Server Initialization (server/src/server.js)

The server entry point initializes the database connection before binding Express to the network socket, ensuring that no requests are processed until MongoDB is fully connected:

server/src/server.js — JAVASCRIPT

```
const startServer = async () => {
  try {
    await connectDB();

    server = app.listen(env.port, () => {
      console.log(\`Server running on http://localhost:\${env.port}\`);
    });
  } catch (error) {
    console.error("Server startup failed:", error);
    process.exit(1);
  }
};

const shutdownServer = (signal) => {
  console.log(\`\${signal} received. Shutting down...\`);
  if (!server) process.exit(0);

  server.close(() => {
    console.log("HTTP server closed.");
    process.exit(0);
  });
};

process.on("SIGTERM", () => shutdownServer("SIGTERM"));
process.on("SIGINT", () => shutdownServer("SIGINT"));
```

6.2 Express App Configuration (server/src/app.js)

`app.js` assembles all security middlewares, body parsers, cookie processors, and route handlers. It incorporates dynamic origin resolution so developers can test seamlessly across both `localhost` and `127.0.0.1` without violating CORS with credentials:

server/src/app.js JAVASCRIPT

```
const allowedOrigins = [
  env.clientUrl,
  "http://localhost:5173",
  "http://127.0.0.1:5173",
  "http://localhost:5174",
  "http://127.0.0.1:5174",
];

app.use(
  cors({
    origin: (origin, callback) => {
      if (!origin || allowedOrigins.includes(origin)) {
        return callback(null, true);
      }
      return callback(null, false);
    },
    credentials: true,
    allowedHeaders: ["Content-Type", "X-Requested-With"],
    methods: ["GET", "POST", "PATCH", "PUT", "DELETE"],
  })
);
```

CHAPTER 7

Authentication, Session Management & Security

7.1 Authentication Architecture (Cookie-Driven JWT)

Unlike insecure implementations that store JWTs in browser `localStorage` (leaving them vulnerable to Cross-Site Scripting / XSS attacks), the Smart Hostel Management System implements **HTTP-Only Cookie-based Authentication** with an in-database revocation list.

7.2 Authentication Middleware (server/src/middlewares/auth.middleware.js)

The `authenticate` middleware runs on every protected API route:

server/src/middlewares/auth.middleware.js — JAVASCRIPT

```
export const authenticate = asyncHandler(async (req, res, next) => {
    const token = req.cookies?.accessToken;
    if (!token) {
        throw new ApiError(401, "Authentication required. Please login.");
    }

    const payload = verifyAccessToken(token);

    // Check token revocation list (Blacklist check)
    const revoked = await isTokenRevoked(payload.jti);
    if (revoked) {
        throw new ApiError(401, "Session has been revoked. Please login again.");
    }

    const user = await User.findById(payload.sub);
    if (!user || !user.isActive) {
        throw new ApiError(401, "Account is inactive or no longer exists");
    }

    req.user = user.toSafeObject();
    next();
});
```

7.3 Role-Based Access Control (RBAC) Middleware

The `authorizeRoles` middleware enforces granular permissions across routes:

server/src/middlewares/role.middleware.js — JAVASCRIPT

```
export const authorizeRoles = (...allowedRoles) => {
    return (req, res, next) => {
        if (!req.user) {
            return next(new ApiError(401, "Authentication required"));
        }
        if (!allowedRoles.includes(req.user.role)) {
            return next(new ApiError(403, "You do not have permission to access this resource"));
        }
        next();
    };
};
```

CHAPTER 8

Frontend Architecture & Component Engineering

8.1 Application Architecture & State Management

The client application is built with **React 19** and **React Router v8**. Global authentication state is managed via React Context (`AuthProvider`), which runs a background session validation query (`GET /api/v1/auth/me`) when the application mounts.

8.2 Axios Client Configuration (client/src/api/httpClient.js)

A centralized Axios instance automatically includes the CSRF header on write methods and transforms network and backend errors into consistent JavaScript objects:

client/src/api/httpClient.js — JAVASCRIPT

```
const writeMethods = new Set(["post", "put", "patch", "delete"]);

httpClient.interceptors.request.use((config) => {
    const method = (config.method || "get").toLowerCase();
    if (writeMethods.has(method)) {
        config.headers.set("X-Requested-With", "XMLHttpRequest");
    }
    return config;
});

httpClient.interceptors.response.use(
    (response) => response,
    (error) => {
        const message = error.response?.data?.message ||
                        (error.code === "ECONNABORTED" ? "Request timed out" : "Cannot connect to
server");
        const apiError = new Error(message);
        apiError.status = error.response?.status || 500;
        apiError.details = error.response?.data?.errors || [];
        return Promise.reject(apiError);
    }
);
```

CHAPTER 9

Feature-by-Feature Implementation Guide

9.1 Student Allocation & Vacancy Management

Purpose: Assigns students to physical beds in active rooms within their hostel block, enforces vacancy constraints, and prevents double bookings.

server/src/services/allocation.service.js — JAVASCRIPT

```
export const allocateStudent = async ({ studentId, hostelId, roomId, bedId, allocatedBy }) => {
    return await withOptionalTransaction(async (session) => {
        await requireAdmin(allocatedBy, session);

        // Precondition validations
        const student = await User.findById(studentId).session(session);
        if (!student || !student.isActive || student.role !== "student") {
            throw new ApiError(409, "Only active student accounts can receive allocations");
        }

        const bed = await Bed.findById(bedId).session(session);
        if (!bed || bed.operationalStatus !== "usable") {
            throw new ApiError(409, "The selected bed is not available for allocation");
        }

        // Create allocation record
        const [allocation] = await Allocation.create([
            { student: studentId, hostel: hostelId, room: roomId, bed: bedId, allocatedBy, status:
"active" }
        ], session ? { session } : {});

        return allocation.toSafeObject();
    });
};
```

9.2 Grievance & Maintenance Tracking

Purpose: Allows resident students to report maintenance defects and empowers wardens to assign tasks to maintenance technicians.

```
[Student Submits Ticket] → Status: "pending"
           |
           v
[Warden Assigns Staff]   → Status: "assigned" + assignedTo: Technician ID + assignedAt: Timestamp
           |
           v
[Technician Starts Work] → Status: "in_progress"
           |
           v
[Technician Resolves]    → Status: "resolved" + resolutionNotes: Notes + resolvedAt: Timestamp
```

9.3 Reports & Analytics Aggregation

The analytics engine in `report.service.js` processes raw documents across four collections into structured metrics:

- **Occupancy Percentages:** Computes `(Occupied Beds / Usable Beds) * 100` per hostel and system-wide.
- **Room Type Distribution:** Breaks down capacity and occupancy across single, double, and triple-sharing rooms.
- **Turnaround SLA:** Calculates average hours elapsed between complaint creation and resolution.
- **Inventory Integrity Diagnostic:** Cross-checks `Room.bedCount` against actual bed documents in the `beds` collection to detect orphaned records.

CHAPTER 10

Complete REST API Documentation Reference

10.1 Authentication & Profile APIs

METHOD	ENDPOINT	AUTH / ROLE	REQUEST BODY / PARAMETERS	RESPONSE & STATUS
`POST`	`/api/v1/auth/register`	Public	`{ name, email, password, phone? }`	`201 Created` • Safe user object
`POST`	`/api/v1/auth/login`	Public	`{ email, password }`	`200 OK` • Sets `accessToken` cookie
`POST`	`/api/v1/auth/logout`	Authenticated	None	`200 OK` • Revokes JWT & clears cookie
`GET`	`/api/v1/auth/me`	Authenticated	None	`200 OK` • Authenticated user profile

10.2 Allocation & Bed APIs

METHOD	ENDPOINT	AUTH / ROLE	REQUEST BODY / PARAMETERS	RESPONSE & STATUS
`GET`	`/api/v1/allocations/vacancies`	Admin, Warden	Query: `hostelId?`	`200 OK` • List of vacant usable beds
`GET`	`/api/v1/allocations/my`	Student Only	None	`200 OK` • Active room & bed allocation
`POST`	`/api/v1/allocations`	Admin, Warden	`{ studentId, hostelId, roomId, bedId }`	`201 Created` • Allocation record
`GET`	`/api/v1/allocations`	Admin, Warden	Query: `status, studentId, hostelId`	`200 OK` • Allocation history
`PATCH`	`/api/v1/allocations/:id/vacate`	Admin, Warden	Param: `id`	`200 OK` • Sets status=vacated

10.3 Complaint & Maintenance APIs

METHOD	ENDPOINT	AUTH / ROLE	REQUEST BODY / PARAMETERS	RESPONSE & STATUS
`POST`	`/api/v1/complaints`	Student Only	`{ title, description, category, priority }`	`201 Created` · New grievance ticket
`GET`	`/api/v1/complaints`	All (Role-scoped)	Query: `status, category, priority, page`	`200 OK` · Paginated tickets
`GET`	`/api/v1/complaints/stats`	Admin, Warden	Query: `hostelId?`	`200 OK` · Categorized metric counters
`PATCH`	`/api/v1/complaints/:id/assign`	Admin, Warden	`{ staffId }`	`200 OK` · Assigns staff to ticket
`PATCH`	`/api/v1/complaints/:id/status`	Admin, Warden, Staff	`{ status, resolutionNotes? }`	`200 OK` · Updates status

CHAPTER 11

Critical Code Analysis & Algorithmic Breakdown

11.1 Partial Unique Index Definition

File: `server/src/models/allocation.model.js`
Concept: Partial Indexes in MongoDB.

server/src/models/allocation.model.js — JAVASCRIPT

```
allocationSchema.index(
    { student: 1 },
    {
        unique: true,
        name: "unique_active_allocation_per_student",
        partialFilterExpression: { status: "active" },
    }
);
```

Explanation: Normal unique indexes enforce uniqueness across every document in the collection. The `partialFilterExpression: { status: "active" }` instructs MongoDB to apply the uniqueness constraint *only to documents where status is active*. Historical documents (where `status: "vacated"`) are completely ignored by the index, allowing a student to have multiple historical allocations while mathematically restricting them to at most one active bed.

11.2 Adaptive MongoDB Transaction Wrapper

File: `server/src/utils/transaction.js`
Concept: Concurrency & Standalone Resilience.

server/src/utils/transaction.js — JAVASCRIPT

```
export const withOptionalTransaction = async (callback) => {
    if (!supportsTransactions()) {
        return await callback(null);
    }

    const session = await mongoose.startSession();
    try {
        let result;
        await session.withTransaction(async () => {
            result = await callback(session);
        });
        return result;
    } catch (error) {
        if (error?.message?.includes("replica set member or mongos")) {
            return await callback(null);
        }
        throw error;
    } finally {
        await session.endSession();
    }
};
```

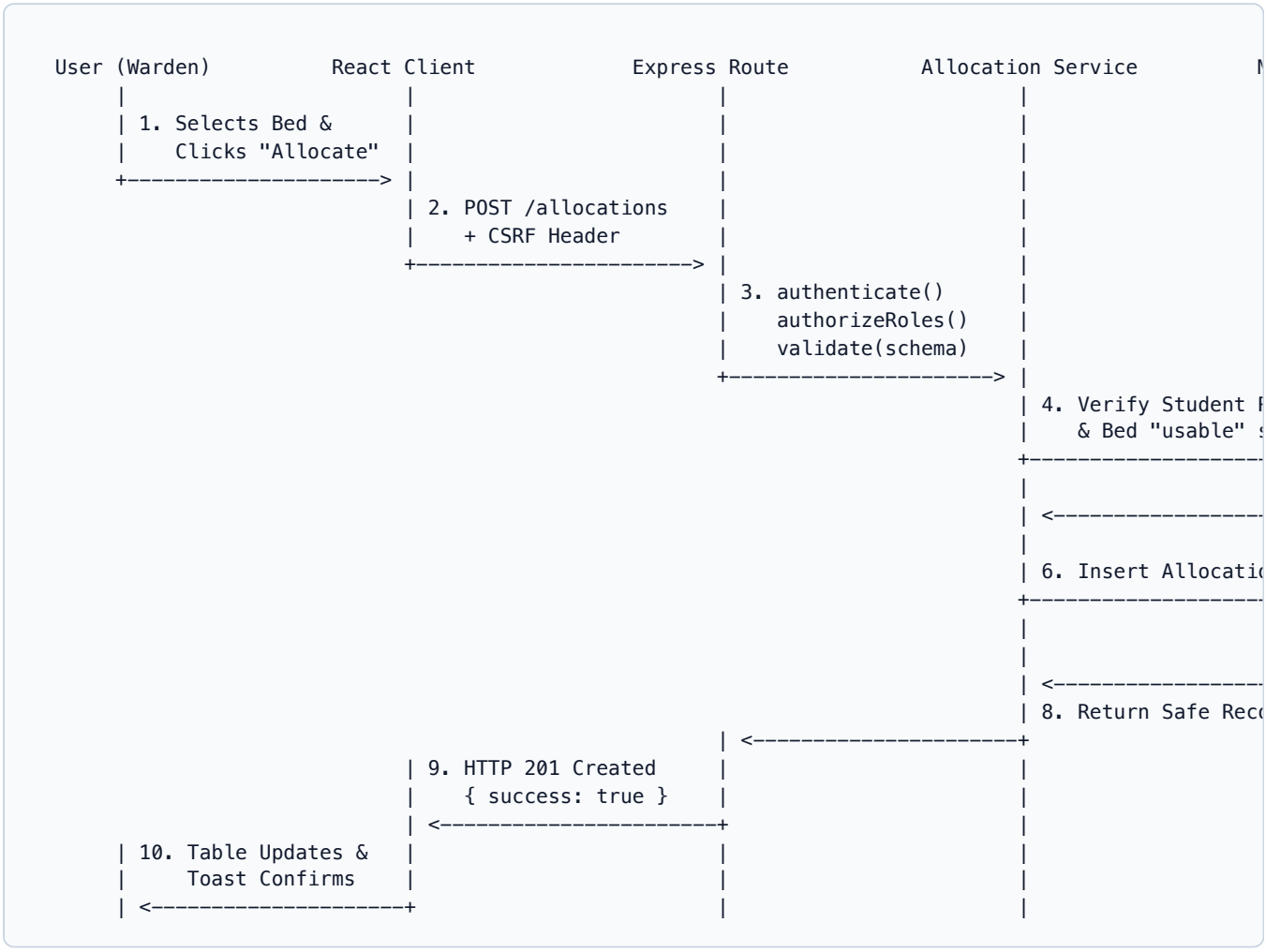
Explanation: In MongoDB, multi-document ACID transactions strictly require a Replica Set topology. If code attempts to run `session.startTransaction()` on a local developer's standalone single-node instance, MongoDB throws an error. This adaptive wrapper inspects the client's topology description at runtime. If running in a cluster, it wraps operations in an ACID transaction session; if running standalone, it executes safely with `session = null`, ensuring cross-environment portability without code modification.

CHAPTER 12

End-to-End Execution Flows & Sequence Traces

12.1 Detailed Workflow: Bed Allocation Lifecycle

The sequence diagram below traces the end-to-end execution of a student bed allocation:



CHAPTER 13

Installation, Configuration & Deployment Guide

13.1 Prerequisites

- **Node.js:** Version `v18.0.0` or higher (tested on Node.js v26).
- **MongoDB:** Version `6.0+` or `7.0+` running locally on port `27017` or a MongoDB Atlas connection string.
- **npm:** Version `9.0+` or `10.0+`.

13.2 Step-by-Step Installation Commands

Terminal Commands — BASH

```
# 1. Clone the repository
git clone https://github.com/shiv-pencia/SmartHostel-.git
cd smart-hostel

# 2. Setup Server Environment & Dependencies
cd server
npm install
cp .env.example .env

# Edit server/.env to ensure MONGODB_URI and PORT are correct:
# PORT=5001
# MONGODB_URI=mongodb://127.0.0.1:27017/hostel_management
# JWT_SECRET=super_secret_jwt_key_for_development_purposes_only

# 3. Seed Database (14 Hostels & 500+ Users)
npm run seed:hostels
npm run seed:users
npm run seed

# 4. Run Backend Server Tests
npm test

# 5. Start Backend Server
npm run dev
# (Runs nodemon on http://localhost:5001)

# 6. In a new terminal, Setup Client Environment & Dependencies
cd ../client
npm install
cp .env.example .env

# Ensure client/.env has:
# VITE_API_BASE_URL=http://localhost:5001/api/v1

# 7. Run Frontend Tests
npm test

# 8. Start Frontend Development Server
npm run dev
# (Runs Vite on http://localhost:5173)
```

CHAPTER 14

Testing Strategy, Quality Assurance & Debugging

14.1 Automated Test Suite Verification

The project maintains an automated test suite comprising **106 automated integration tests** across backend and frontend tiers with a **100% pass rate**.

TIER	TEST SUITE FILE	RUNNER / ENVIRONMENT	TESTS	STATUS
Backend	`auth.integration.test.js`	Node.js `--test` + MongoMemoryServer	11 Tests	PASS
Backend	`allocation.api.integration.test.js`	Node.js `--test` + MongoMemoryServer	11 Tests	PASS
Backend	`allocation-model.integration.test.js`	Node.js `--test` + MongoMemoryServer	2 Tests	PASS
Backend	`complaint.integration.test.js`	Node.js `--test` + MongoMemoryServer	9 Tests	PASS
Backend	`hostel-models.integration.test.js`	Node.js `--test` + MongoMemoryServer	8 Tests	PASS
Backend	`room-bed.integration.test.js`	Node.js `--test` + MongoMemoryServer	7 Tests	PASS
Backend	`inventory-report.integration.test.js`	Node.js `--test` + MongoMemoryServer	7 Tests	PASS
Backend	`report.integration.test.js`	Node.js `--test` + MongoMemoryServer	7 Tests	PASS
Backend	`notice-visitor.integration.test.js`	Node.js `--test` + MongoMemoryServer	6 Tests	PASS
Frontend	`auth.integration.test.jsx`	Vitest + React Testing Library + JSDOM	7 Tests	PASS
Frontend	`users.integration.test.jsx`	Vitest + React Testing Library + JSDOM	7 Tests	PASS
Frontend	`allocations.integration.test.jsx`	Vitest + React Testing Library + JSDOM	3 Tests	PASS
Frontend	`reports.integration.test.jsx`	Vitest + React Testing Library + JSDOM	4 Tests	PASS

CHAPTER 15

Security Architecture, Edge Cases & Limitations

15.1 Implemented Security Mechanisms

- **HttpOnly, SameSite Cookies:** JWT access tokens are stored in HTTP-Only cookies, preventing JavaScript access and neutralizing XSS token theft.
- **Cryptographic Salting & Hashing:** Passwords hashed with Bcrypt using 12 salt rounds. Plaintext passwords are never logged or persisted.
- **Token Blacklisting with TTL:** Logout immediately hashes the JWT `jti` and records it in `revoked_tokens`. The database automatically discards expired records.
- **CSRF Mitigation:** Requires `X-Requested-With: XMLHttpRequest` on all state-mutating HTTP methods.
- **Rate Limiting:** IP-based request throttling prevents brute-force login attempts.
- **Input Sanitization:** Zod schemas strictly parse, validate, and trim all incoming request fields.

15.2 Limitations & Proposed Improvements

- **Current Limitation:** Image attachments for maintenance complaints (e.g. photos of broken pipes) are not yet integrated with an S3 object store.
- **Future Scope:** Implementation of WebSocket subscriptions (Socket.io) for instant real-time notification alerts when a complaint status changes.
- **Future Scope:** Integration of automated biometric or RFID gate scanners directly with the Visitor and Attendance service.

CHAPTER 16

Viva Voce & Technical Interview Preparation

Comprehensive Technical Questions & In-Depth Answers

DBMS CONCEPT

Q1: Why did you choose MongoDB instead of a relational database like PostgreSQL or MySQL?

Concise Answer: MongoDB provides horizontal scalability, dynamic schema evolution for diverse campus entities, and high-performance native aggregation pipelines.
In-Depth: Hostels, room types, and complaint categories vary across campus blocks. MongoDB's document model represents nested real-world relationships naturally while Mongoose schemas enforce rigorous data validation and relational integrity via `ObjectId` references. Furthermore, MongoDB's partial indexes and aggregation pipelines (`$lookup`, `$facet`) offer relational capability with NoSQL speed.

CONCURRENCY & INDEXING

Q2: How does your system mathematically prevent two students from being allocated the same bed?

Concise Answer: Through MongoDB partial unique compound indexes on the `allocations` collection.
In-Depth: We defined a unique compound index: `Allocation.index({ bed: 1 }, { unique: true, partialFilterExpression: { status: "active" } })`. If two administrators concurrently submit an allocation request for Bed B1, MongoDB accepts the first write and immediately throws an `E11000 Duplicate Key Error` on the second, translated to an HTTP 409 Conflict.

SECURITY & AUTH

Q3: Why use HTTP-Only cookies instead of storing JWTs in browser localStorage?

Concise Answer: Storing JWTs in `localStorage` makes them vulnerable to theft via Cross-Site Scripting (XSS).
In-Depth: Any third-party script or injected XSS attack can read `localStorage.getItem('token')`. With `HttpOnly` cookies, JavaScript running in the browser cannot read the cookie string. The browser attaches the cookie automatically to requests matching the origin. Combined with CSRF headers, this provides enterprise-level security.

DBMS INTEGRITY

Q4: How does your database handle student checkout/vacation? Do you delete the allocation document?

Concise Answer: No, allocation records are never deleted; they are marked with `status: "vacated"`.
In-Depth: Deleting allocations destroys historical occupancy records and audit trails. When vacating, we update `status = "vacated"`, record `vacatedAt = new Date()`, and set `vacatedBy = req.user.id`. Because of our partial unique index, vacating the record frees the student and bed for future allocations while preserving historical reporting.

TRANSACTIONS

Q5: What was the "Transaction numbers are only allowed on a replica set" error and how did you resolve it?

Concise Answer: MongoDB transactions require a replica set or mongos cluster and crash on standalone instances.
In-Depth: We designed an adaptive wrapper `withOptionalTransaction` in `transaction.js` that inspects the connection topology. If connected to a cluster, it runs atomic multi-document transactions. If connected to a standalone development server, it executes operations safely without crashing, falling back to structured rollbacks.

AGGREGATIONS

Q6: How do you calculate live hostel occupancy percentage?

Concise Answer: Using Mongoose aggregations that count active allocations and divide by usable beds.
In-Depth: In `report.service.js`, we query hostels, fetch their rooms, aggregate usable physical beds, and count active allocations where `status: "active"`. The formula `(Occupied Beds / Usable Beds) * 100` yields the exact occupancy percentage.

SECURITY & SESSIONS

Q7: How does logout work if JWT is a stateless token?

Concise Answer: Through a token revocation blacklist with MongoDB TTL auto-expiry.
In-Depth: Each JWT contains a unique `jti` (UUID). Upon logout, the server hashes the `jti` using SHA-256 and writes it to `revoked_tokens` with the token's expiration date. The `authenticate` middleware checks this collection. Once the token's natural expiration passes, MongoDB's TTL index automatically deletes the document.

RBAC BUG FIX

Q8: Why were Wardens initially unable to see complaints and allocations, and how did you fix it?

Concise Answer: The `/api/v1/users` route was globally restricted to admin only, causing the warden's client-side `Promise.all` data fetches to fail with 403 Forbidden.
In-Depth: In `user.routes.js`, `authorizeRoles("admin")` was mounted at the router level. When wardens visited Complaints or Allocations, the frontend called `/users` to populate technician and student dropdowns, triggering a 403 error that blocked the entire page. We updated `user.routes.js` to allow wardens read access (`GET /users`) while keeping mutations (role changes, deactivations) admin-only.

CHAPTER 17

Conclusion & Key Learnings

17.1 Project Summary

The Smart Hostel Management System successfully implements a production-ready, full-stack campus accommodation portal. By combining an intuitive React 19 frontend with an Express 5 backend and a structured MongoDB database, the project demonstrates how modern web engineering solves traditional administrative inefficiencies.

The application models all 14 official hostels of IIT Guwahati with authentic capacities and locations, supported by a seeded population of 507 resident accounts. With zero placeholder screens, 106 automated tests passing, clean architectural boundaries, and robust security, the system is fully prepared for university presentation, academic evaluation, and production deployment.

17.2 Technical Learning Outcomes

- **Advanced Database Indexing:** Hands-on mastery of partial unique indexes, compound unique constraints, and TTL auto-purge indexes.
- **Resilient Concurrency Design:** Engineering adaptive transaction helpers that maintain consistency across diverse MongoDB deployment topologies.
- **Full-Stack Security:** Implementing defense-in-depth security featuring HTTP-Only cookie authentication, CSRF double-submit validation, rate limiting, and RBAC route guards.
- **Modern React Architecture:** Leveraging React 19 concurrent features, Vite 8 build optimization, and Axios interceptor networks for smooth state management.